

UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI MATEMATICA

Laurea Triennale in Informatica

Anno Accademico 2024/2025

PROGETTO DI PROGRAMMAZIONE AD
OGGETTI

Gestionale Biblioteca

Alessandro Frison

Matricola: 2111932

Indice

1	Introduzione	2
2	Descrizione del modello	2
2.1	Model	2
2.2	View	4
2.2.1	Media Manager	4
2.2.2	Loan Manager	5
3	Polimorfismo	5
4	Persistenza dei dati	6
5	Funzionalità implementate	7
6	Rendicontazione delle ore	8
7	Conclusioni	8

1 Introduzione

Il progetto presentato consiste in un'applicazione sviluppata in **C++**, con interfaccia grafica realizzata tramite il **framework Qt**, sfruttando in particolare il modulo **QWidget**. L'obiettivo dell'applicazione è la gestione di una biblioteca fisica contenente diversi tipi di media, tra cui **libri**, **film** e **articoli**.

Un singolo media può essere presente in più copie fisiche, ognuna identificata da un codice univoco. Questa scelta riflette il funzionamento reale di una biblioteca, dove lo stesso titolo può essere disponibile in più esemplari. A tal fine, sono stati implementati tutti i metodi fondamentali per la gestione dei dati (CRUD): è possibile **aggiungere** nuovi media, **modificarne** le informazioni, **eliminarli** dal sistema e **consultarli**.

L'applicazione supporta inoltre l'importazione e il salvataggio di file **JSON**, sia vuoti che contenenti dati salvati in precedenza, rendendo possibile la persistenza dello stato della biblioteca tra diverse sessioni. Il sistema consente anche di registrare i prestiti dei media e la loro successiva restituzione.

Ogni categoria di media presenta un insieme di attributi comuni e altri specifici, che vengono visualizzati dinamicamente in base alla tipologia selezionata dall'utente.

2 Descrizione del modello

Il progetto è stato sviluppato seguendo il pattern software architetturale **Model-View**, il quale ha permesso una netta separazione tra la logica di accesso e rappresentazione dei media (il Model) e la logica di visualizzazione e interazione con i media (la View).

2.1 Model

La Figura 1 illustra la struttura delle classi principali appartenenti al *Model* dell'applicazione, rappresentate secondo lo standard **Unified Modeling Language (UML)**. L'architettura logica del modello si fonda su una gerarchia che ha origine da una classe astratta denominata *Media*, la quale racchiude gli **attributi comuni** e definisce **metodi virtuali puri**, destinati a essere implementati dalle relative sottoclassi. Le principali derivazioni della classe base sono le seguenti:

- **Libro**: comprende gli attributi specifici *autore*, *editore*, *numero di pagine* e *codice ISBN*;
- **Film**: definito tramite *regista*, *durata* e *cast*;
- **Articolo**: caratterizzato da *autore*, *rivista*, *volume* e *numero di pagine*.

Ogni sottoclasse presenta una propria configurazione di attributi specifici e fornisce una **ridefinizione personalizzata dei metodi astratti** ereditati dalla classe madre. Tutte le entità dispongono di metodi *getter* e *setter*, conformemente ai principi di **incapsulamento** e **protezione dei dati**. Un componente cruciale del modello è rappresentato dalla classe **MediaRepo**, responsabile della **gestione in memoria** dei dati e della loro **persistenza** su file in formato JSON. Essa offre funzionalità per la **creazione**, **modifica** e **rimozione** delle

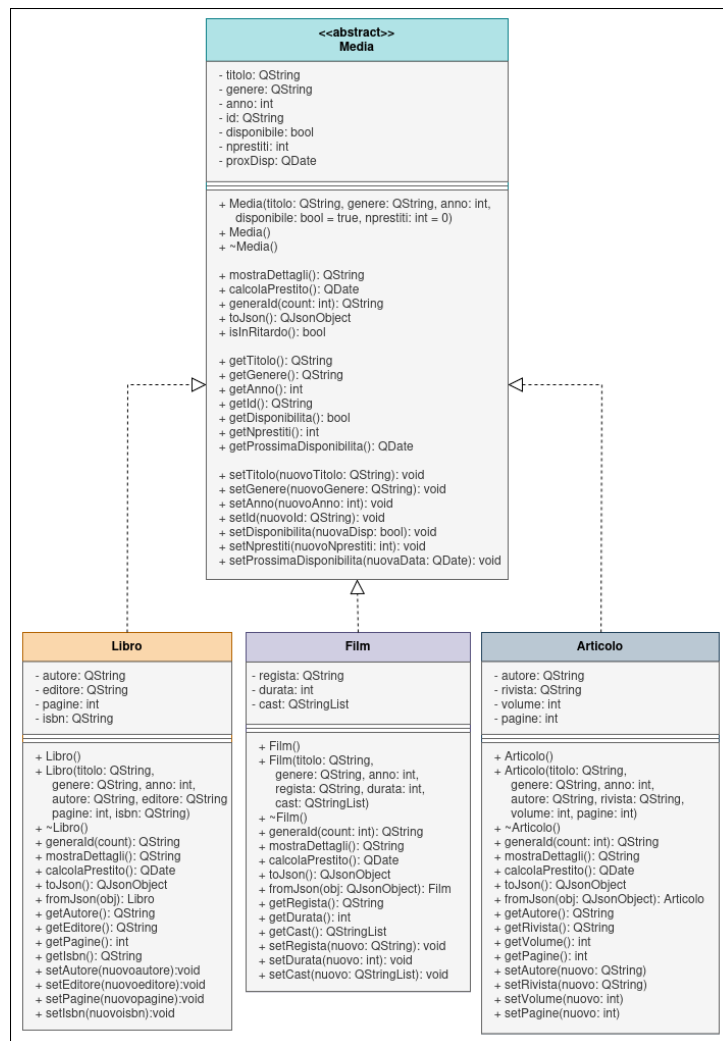


Figura 1: Diagramma delle classi principali del progetto

istanze di media, nonché per il **salvataggio e caricamento dei dati** da e verso il file JSON. In questo modo, i componenti dell'interfaccia utente (View) possono delegare a tale classe la gestione dinamica delle collezioni di media. Infine, la classe **MediaRepo** implementa anche la logica per la gestione delle operazioni di **prestito** e **restituzione**, fungendo da strato intermedio tra la logica dell'applicazione e l'interfaccia utente.

2.2 View

La finestra principale dell'applicazione è definita all'interno del widget `MainWindowWidget`. Questo componente rappresenta la struttura centrale dell'interfaccia grafica e consente la navigazione tra le sue due principali sottosezioni.

2.2.1 Media Manager

Il widget `MediaManager` costituisce il fulcro operativo dell'interfaccia utente per quanto riguarda la **gestione dei media**. In esso sono implementate tutte le funzionalità necessarie per l'**inserimento**, la **modifica** e la **cancellazione** di oggetti multimediali. Inoltre, vengono fornite le opzioni per **importare** ed **eliminare** un archivio in formato JSON. L'interfaccia grafica è suddivisa in quattro aree funzionali principali:

- **Barra di ricerca (in alto):** consente di eseguire ricerche filtrando i media in base al *titolo* o all'*autore*. L'inserimento di una stringa permette di effettuare un confronto parziale, migliorando la flessibilità della ricerca.
- **Lista dei media (a sinistra):** visualizza l'elenco completo dei media registrati. Ogni elemento è rappresentato da un'*icona identificativa* in base alla tipologia (libro, film o articolo), con il **titolo evidenziato in grassetto**, seguito dal nome dell'autore. A seconda del tipo di media, viene mostrato un attributo aggiuntivo (ISBN per i libri, durata per i film, nome della rivista per gli articoli). Infine, una riga supplementare riporta un'emoji e una stringa descrittiva del tipo di media.
- **Form di inserimento e modifica (a destra):** permette di compilare i campi relativi a un nuovo media da aggiungere. Tutti i campi sono editabili, ad eccezione dell'**ID**, che viene generato automaticamente al momento della creazione. Quando un media viene selezionato dalla lista, il form si popola automaticamente con i suoi dati, permettendo anche la **modifica** o la **visualizzazione**. L'interfaccia prevede un *radio button* per la selezione della tipologia di media, che rende visibili i campi specifici in base alla scelta effettuata.
- **Pannello dei comandi (in basso a destra):** ospita una pulsantiera composta da sei pulsanti che costituiscono il centro funzionale dell'applicazione. Le azioni disponibili sono:
 - **Aggiunta** di un nuovo media;
 - **Eliminazione** di un media selezionato;
 - **Modifica** di un media esistente;
 - **Importazione** di un archivio JSON;
 - **Cancellazione** del contenuto di un archivio JSON;
 - **Reset** dei campi del form.

2.2.2 Loan Manager

Il widget **LoanManager** è dedicato alla gestione delle operazioni di **prestito** e **restituzione** dei media. L'interfaccia è suddivisa in tre componenti funzionali principali:

- **Barra di ricerca e filtri (in alto)**: consente di cercare i media per *titolo* o *autore*, anche parzialmente. Inoltre, è possibile applicare un filtro in base allo stato del media, selezionando tra le seguenti categorie: *prestiti in corso*, *prestiti scaduti*, *disponibili al prestito* o *tutti i media*.
- **Elenco dei media filtrati (parte centrale)**: mostra i risultati della ricerca applicando i criteri selezionati. La visualizzazione dei singoli media è analoga a quella presente nel **MediaManager**, con l'aggiunta di una codifica cromatica sul titolo:
 - **verde** per i media disponibili;
 - **arancione** per quelli attualmente in prestito;
 - **rosso** per i media il cui prestito risulta scaduto.
- **Pulsanti di azione (in basso)**: sono presenti due pulsanti principali. Il primo consente di **effettuare un prestito** per il media selezionato, calcolando automaticamente la *data di prossima disponibilità* in base alla sua tipologia. Il secondo pulsante consente la **restituzione** del media selezionato, aggiornandone lo stato e rendendolo nuovamente disponibile.

3 Polimorfismo

L'applicazione sfrutta il concetto di **polimorfismo** per garantire un comportamento differenziato e specifico nei metodi comuni alle varie tipologie di media. In particolare, sono stati implementati metodi virtuali nella classe base astratta, successivamente sovrascritti nelle classi derivate, risultando fondamentali per il corretto funzionamento del sistema. Tra i metodi polimorfici principali si evidenziano:

- **generaId()**: questo metodo consente la generazione di un **identificativo univoco** per ciascun media, adattandosi dinamicamente in base alla tipologia dell'oggetto. In tal modo si garantisce la coerenza e l'univocità all'interno della collezione.
- **calcolaPrestito()**: implementato in modo specifico in ciascuna classe derivata, consente di determinare la **durata del prestito** prevista per ogni media. Il sistema prevede:
 - *30 giorni* per un libro;
 - *7 giorni* per un film;
 - *2 giorni* per un articolo.

Questo comportamento differenziato è possibile proprio grazie all'uso del polimorfismo.

- **Metodi per la serializzazione JSON:** ogni classe implementa metodi per la *lettura* e la *scrittura* dei propri dati da e verso il formato JSON. Questo approccio consente una gestione modulare e flessibile della **persistenza dei dati**, facilitando le operazioni di salvataggio e caricamento dell'archivio.
- **Metodi per la visualizzazione** dei media nelle varie liste. In particolare, sono stati implementati tre metodi specifici:
 - `getTipoIcona()`: restituisce il tipo del media accompagnato da un'emoji rappresentativa.
 - `getDettagliString()`: restituisce una stringa contenente le informazioni salienti del media, utili per una descrizione sintetica ma completa.
 - `getIconaPath()`: restituisce il path dell'icona corrispondente al tipo di media, al fine di rendere immediatamente riconoscibile la categoria nella lista dei media.

4 Persistenza dei dati

La **persistenza dei dati** all'interno dell'applicazione è gestita tramite il formato standard JSON. Il sistema utilizza una *path predefinita* per il caricamento del file all'avvio: nel caso in cui tale percorso non contenga un file valido, l'applicazione si avvia senza caricare alcun contenuto. In assenza di un file JSON conforme, non è consentita alcuna operazione di gestione dei media fino all'importazione manuale di un archivio valido. È tuttavia possibile importare un file JSON tramite l'apposita funzionalità integrata nell'interfaccia grafica, con la possibilità di *ripristinare* l'archivio in qualsiasi momento. Il sistema implementa un controllo di **validità del formato**: in caso di file JSON non conforme alla struttura prevista, il caricamento viene bloccato. In questo modo si previene l'inserimento di dati corrotti o incompatibili con il modello dati dell'applicazione. Ogni operazione che comporta una modifica della memoria dinamica (aggiunta, rimozione o modifica di un media) viene immediatamente riflessa nel file JSON, assicurando la *sincronia tra lo stato in memoria e quello persistente*. Ciò consente di recuperare il lavoro svolto in caso di arresto improvviso o riavvio dell'applicazione, semplicemente reimportando il file utilizzato in precedenza. Il file JSON è strutturato come una **lista di oggetti**, suddivisi per tipologia di media: ciascuna categoria (libri, film, articoli) dispone di una propria lista interna, contenente tutti gli attributi specifici delle rispettive classi. Infine, all'interno della directory del progetto è presente una cartella denominata **Database**, la quale include alcuni *file JSON di esempio* pronti all'uso per facilitare i test e le dimostrazioni.

5 Funzionalità implementate

Il progetto sviluppato prevede una serie di **funzionalità** mirate a garantire un'esperienza d'uso completa, efficiente e coerente con le esigenze di un gestionale bibliotecario. Le principali funzionalità realizzate sono le seguenti:

- **Gestione di tre tipologie di media** (*libro, film, articolo*) tramite operazioni di tipo **CRUD** (*Create, Read, Update, Delete*), accessibili attraverso un'interfaccia intuitiva.
- **Persistenza dei dati** attraverso l'utilizzo del formato **JSON**, con supporto sia alla lettura sia alla scrittura, al fine di garantire la conservazione dei dati tra sessioni differenti.
- **Scorciatoie da tastiera** per ottimizzare l'interazione con l'applicazione e velocizzare le operazioni più comuni.
- **Utilizzo di icone intuitive** per identificare rapidamente la tipologia di ogni media nella lista visualizzata.
- **Visualizzazione dei media** sotto forma di *lista interattiva*, con elementi cliccabili e dinamici che ne mostrano lo stato e le caratteristiche salienti.
- **Dettagli contestualizzati** per ciascun media, adattati in base alla tipologia selezionata e mostrati all'interno di un form dedicato.
- **Interfaccia responsiva**: finestre e widget si adattano automaticamente al ridimensionamento, garantendo un layout coerente anche su schermi di dimensioni differenti.
- **Navigazione centralizzata**: tutte le sezioni dell'applicazione sono accessibili dalla finestra principale, minimizzando l'utilizzo di *finestre modali* o *pop-up*.
- **Controllo automatico dell'unicità degli ID**, assicurando che ogni media gestito nel sistema sia identificato in maniera univoca.
- **Gestione completa dei prestiti**, con calcolo automatico della scadenza in base alla tipologia del media e funzionalità di restituzione.
- **Sistema di filtro avanzato** che consente ricerche personalizzate in tutte le sezioni dell'applicazione, sulla base di criteri specifici come titolo, autore o stato del prestito.
- **Controllo del tema** all'avvio dell'applicazione, che consente di selezionare temi grafici differenti. A seconda del tema scelto, vengono modificati i colori della barra di ricerca e delle icone utilizzate nell'interfaccia.

6 Rendicontazione delle ore

Attività	Ore previste	Ore effettuate	Note
Progettazione	6	7	Richiesto maggior tempo del previsto per un approfondimento completo di tutti gli aspetti progettuali.
Sviluppo Model	11	15	Riscontrate difficoltà nell'implementazione della classe <i>MediaRepo</i> .
Sviluppo View	12	18	Ritardi dovuti allo studio approfondito delle funzionalità offerte da Qt.
Testing	6	5	Fase conclusa senza particolari problematiche.
Documentazione	5	5	Attività svolta secondo la pianificazione iniziale.
Totale	40	50	

La stima iniziale delle ore di lavoro è stata superata principalmente a causa del tempo necessario per approfondire la documentazione di Qt, in particolare riguardo a funzionalità e costrutti avanzati, come l'utilizzo degli *smart pointer*, che hanno successivamente semplificato notevolmente lo sviluppo. La fase di sviluppo del *Model* non ha presentato particolari difficoltà, contrariamente alla fase di sviluppo della *View*, che ha richiesto uno studio più dettagliato di tutti gli elementi grafici messi a disposizione da Qt. Per le restanti attività non si sono riscontrati problemi rilevanti, grazie a un'accurata progettazione iniziale che ha consentito di organizzare il lavoro in modo efficiente e sistematico.

7 Conclusioni

In conclusione, questo progetto si è rivelato un elemento **fondamentale** e **integrante** del percorso formativo, offrendo un'importante occasione per approfondire costrutti avanzati del linguaggio *C++*, non affrontati durante le lezioni. Inoltre, ha consentito di consolidare le conoscenze pregresse in modo significativo. L'iniziale difficoltà nell'approccio al codice *da zero* è stata superata rapidamente con l'implementazione delle classi di base, che ha facilitato lo sviluppo delle funzionalità successive, rendendo il lavoro stimolante e gratificante. Ulteriore valore è stato acquisito attraverso l'esplorazione e l'utilizzo di **Qt Creator**, strumento recentemente richiesto nel mio ambito lavorativo, rafforzando così anche competenze tecniche spendibili professionalmente.